

Iterative Problem Solving

Phi
Institute of
Design

2026
Course
Profile

© 2026 Phi Institute of Design. All Rights Reserved.
This content is protected by copyright law.
Unauthorized reproduction, distribution, or transmission
of any part of this material, in any form or by any means,
is strictly prohibited without prior written permission.



A. 전문가 소개

박종민

시장에서 동작하는 디자인을 추구하며 생존이 걸린 치열한 환경에서 다양한 임팩트를 내왔다. IT 스타트업계에서 18년간 프리랜서부터 유니콘급 스타트업 헤드 오브 디자인까지 경험했으며, 현재는 디자인 나침반을 운영하며 비즈니스 임팩트를 내는 디자인을 연구하고 있다.

정신재

대기업과 유니콘 스타트업을 오가며 빅테크부터 커머스, 콘텐츠까지 다양한 영역의 프로젝트를 런칭해왔다. 디자이너는 화면을 구현하는 사람 그 이상이라는 신념을 바탕으로, 디자이너들이 디자인의 실천적 가치를 통해 비즈니스와 세상에 실질적 임팩트를 만들어낼 수 있도록 교육에 매진하고 있다.

B. 수업 개요

처음 발견한 문제의 해결책은 사용자의 문제를 해소하지 못할 때가 많다. 이 수업에서는 표면적인 현상 너머의 근본 원인을 집요하게 파고들어, 해결책을 만들어 검증할 때마다 문제를 더 깊이 다시 정의한다. 문제의 해상도를 높여가며 가치 있는 문제에 도달하고, 그것을 디자인으로 해결하는 법을 체화하게 된다.

수업 전체는 하나의 사이클을 반복하는 것으로 이루어진다. 사용자의 불편함을 포착해 문제를 정의하고, 가능한 해결책을 폭넓게 탐색해 그중 문제와 정확히 맞물린 것을 프로토타입으로 만들고, 문제가 실제로 해결되는지 검증한다. 그리고 여기서 멈추지 않는다. 검증 결과에서 더 근본적인 원인이 드러나면 문제를 다시 정의하고, 사이클을 처음부터 다시 돈다. 해결책을 한 번 만들어 보는 것이 아니라, 하나의 문제를 여러 바퀴에 걸쳐 파고드는 것이 이 수업의 기본 단위다.

C. 학습 목표

Before

- 사용자가 처한 문제 상황을 구체적으로 상상하지 못한다.
- 문제 상황에서 겉으로 드러나는 증상 그대로를 문제로 정의한다.
- 해결책을 먼저 떠올리고, 정의해둔 문제에 끼워 맞추는 결과물을 만든다.
- 결과물을 만들면 끝났다고 여기고, 문제가 실제로 해결됐는지 확인하지도, 다시 정의하지도 않는다.

After

- 사용자의 불편함을 예민하게 감지하고, 그것이 누구에게 어떤 상황에서 일어나는지 장면 단위로 생생하게 서술할 수 있다.
- 증상과 문제를 구분하고, 그 문제가 발생하는 근본 원인 (root cause)이 무엇인지 논리적으로 설명할 수 있다.
- 문제와 정확히 연결되어, 사용자가 쓰는 순간 문제가 풀렸다고 느끼는 해결책을 만들 수 있다.
- 프로토타입으로 문제가 실제로 해결되는지 검증하고, 그 결과에서 더 근본적인 원인을 발견해 문제를 다시 정의할 수 있다.

D. 주차별 과제

매주 문제 정의 → 솔루션 정의 → 프로토타입 제작 → 검증 사이클을 반복한다. 15주는 네 구간으로 나뉘며, 구간마다 주제와 검증 방법, 제출하는 산출물이 달라진다. 아래 표에서 이번 주차 속한 구간을 확인하고, 구간별 설명과 E. 과제 상세를 참고한다

주차	사용자	문제 및 솔루션 검증 방법	1 Pager		프로토타입	공개 게시물
			Docs ver.	Web ver.		
Warm-up~3주차	동료 학습자	동료 학습자의 피드백	O	X	O	X
4~7주 차	직접 정의	실제 사용자 인터뷰	O	X	X	X
8~11주 차	직접 정의	프로토타입 직접 사용	O	X	O	X
12~15주 차	직접 정의	SNS 공개 및 서비스 개발사 제안	X	O	O	O

Warm up~3주차 동료 학습자의 불편함에서 좋은 문제를 골라내는 기준을 익힌다.	동료 학습자가 자주 사용하는 서비스 하나를 고르고 불편함의 근본 원인을 찾아 해결하는 인터페이스를 만든다. 서로의 작업물 중 문제를 가장 잘 해결한 사례에 투표하며 무엇이 좋은 문제인지 토론한다.
3주차가 지나면	☒ 사용자의 불편함을 예민하게 감지하고, 그것이 누구에게 어떤 상황에서 일어나는지 장면 단위로 생생하게 서술할 수 있다. ☐ 증상과 문제를 구분하고, 그 문제가 발생하는 근본 원인(root cause)이 무엇인지 논리적으로 설명할 수 있다. ☐ 문제와 정확히 연결되어, 사용자가 쓰는 순간 문제가 풀렸다고 느끼는 해결책을 만들 수 있다. ☐ 프로토타입으로 문제가 실제로 해결되는지 검증하고, 그 결과에서 더 근본적인 원인을 발견해 문제를 다시 정의할 수 있다.
4~7주 차 문제를 정의하고 실제 사용자에게서 검증한다.	개선하고 싶은 기존 서비스를 하나 고르고, 그 서비스의 실제 사용자를 매주 인터뷰한다. 내가 상상한 문제가 아니라 사용자가 실제로 겪는 문제인지 매 사이클마다 확인한다.
7주차가 지나면	☒ 사용자의 불편함을 예민하게 감지하고, 그것이 누구에게 어떤 상황에서 일어나는지 장면 단위로 생생하게 서술할 수 있다. ☒ 증상과 문제를 구분하고, 그 문제가 발생하는 근본 원인(root cause)이 무엇인지 논리적으로 설명할 수 있다. ☐ 문제와 정확히 연결되어, 사용자가 쓰는 순간 문제가 풀렸다고 느끼는 해결책을 만들 수 있다. ☐ 프로토타입으로 문제가 실제로 해결되는지 검증하고, 그 결과에서 더 근본적인 원인을 발견해 문제를 다시 정의할 수 있다.

D. 주차별 과제

8~11주 차 솔루션을 고도화하며 문제를 다시 정의한다.	사용자의 미세한 불편까지 해결하도록 솔루션의 완성도를 높인다. 개선한 프로토타입을 직접 사용해 보고, 그 결과에서 더 근본적인 원인이 드러나면 문제를 다시 정의한다.
11주차가 지나면	✔ 사용자의 불편함을 예민하게 감지하고, 그것이 누구에게 어떤 상황에서 일어나는지 장면 단위로 생생하게 서술할 수 있다. ✔ 증상과 문제를 구분하고, 그 문제가 발생하는 근본 원인(root cause)이 무엇인지 논리적으로 설명할 수 있다. ✔ 문제와 정확히 연결되어, 사용자가 쓰는 순간 문제가 풀렸다고 느끼는 해결책을 만들 수 있다. ✔ 프로토타입으로 문제가 실제로 해결되는지 검증하고, 그 결과에서 더 근본적인 원인을 발견해 문제를 다시 정의할 수 있다.
12~15주 차 솔루션의 효용을 입증하고 서비스 개발사에 제안한다.	서비스 개발사를 설득할 웹사이트를 제작한다. 매주 개선한 버전을 공개 게시물로 SNS에 공개해 이해관계 없는 사용자의 반응을 얻고, 그 반응을 근거로 제안 자료를 완성한다. 15주 차 수업이 끝나면 서비스 개발사에 제안하고, 그 결과를 마지막 수업에서 공유한다.

E. 과제 상세

이 섹션은 매주 제출하는 산출물을 어떻게 만드는지 설명한다. 주차별 테이블에서 이번 주 산출물을 확인한 뒤, 해당 항목을 찾아 참고하면 된다.

1 Pager (docs ver.)

<https://docs.google.com/document/>

[d/1zdDIgum9i_57FXLXJtD4MnfzRVMJZVTvBmdTFD3Okws/edit?usp=sharing](https://docs.google.com/document/d/1zdDIgum9i_57FXLXJtD4MnfzRVMJZVTvBmdTFD3Okws/edit?usp=sharing)

지금 문제를 어디까지 봤는지 한 장으로 정리하는 문서. 매달 달라지는 문서 구성은 위 가이드라인에서 확인한다. 매주 새 버전을 쓸 때마다 버전 로그를 남긴다.

- ~3주 차 : 맥락을 전혀 모르는 사람이 읽어도 문제가 발생하는 장면을 그릴 수 있게 쓰는 데 집중한다. 그 문제를 해결하는 솔루션과, 해결됐을 때 무엇이 달라지는지를 읽는 사람이 공감할 수 있도록 작성한다. 특히 첫 줄은 동료 학습자가 내 프로토타입 링크를 누르게 만드는 것이 목표다. 최대한 많은 동료 학습자가 링크를 누르고 싶어지는 문장을 쓴다.
- ~7주 차 : 사용자 인터뷰 내용을 근거로 기존 서비스의 문제를 날카롭게 짚어내는 데 집중한다. 인터뷰 전에 사용자의 문제라고 가정했던 것이 실제로도 문제였는지, 검증한 결과가 드러나도록 작성한다. 인터뷰 과정에서 문제를 다시 정의하는 데 도움이 되는 정보를 얻었다면 반영한다.
- ~11주 차 : 검증 결과를 근거로 문제를 다시 정의하고 솔루션을 고도화한 과정이 드러나게 쓰는 데 집중한다. 다시 정의한 문제가 이전 문제의 원인 계보에서 어떻게 이어지는지, 채택하지 않은 대안과 그 기각 근거는 무엇인지 더 구체적으로 적는다.

1 Pager (web ver.)

서비스 개발사를 설득하기 위한 제안서. docs ver.에서 정리해 온 문제와 솔루션을 웹사이트로 옮겨, 방문자가 읽는 데서 그치지 않고 솔루션을 직접 써 볼 수 있게 만든다.

첫 화면만 보고도 서비스 개발사 담당자가 스크롤하고 싶어지는지가 기준이다. 어떤 구성이 제안을 가장 설득력 있게 만드는지 직접 결정해 웹사이트를 제작하고, 매주 새 버전을 공개 게시물로 공개해 이해관계 없는 사용자의 반응을 얻는다. 그 반응을 근거로 구성을 고쳐 가며, 15주 차에는 서비스 개발사에 제안해 설득할 수 있는 수준까지 완성도를 끌어올린다.

E. 과제 상세

프로토타입

1 Pager에서 설계한 솔루션을 사용자의 입력에 실제로 반응하는 형태로 구현한 산출물. 프로토타이핑 도구(Figma, ProtoPie 등)로 만든 인터랙티브 프로토타입이거나, 클로드 코드 등으로 구현해 실제로 작동하는 앱이어야 한다.

기준은 사용자가 써 보는 순간, 1 Pager에서 정의한 문제가 풀리는 경험을 하느냐다. 화면이 예뻐도 그 순간이 없으면 검증에 쓸 수 없고, 거칠어도 그 순간이 있으면 검증에 쓸 수 있다. 문제와 관계없는 화면은 만들지 않는다. 사용자가 문제를 겪는 그 지점만 정확히 동작하게 만들고, 나머지는 필요한 만큼만 채운다.

공개 게시물

자신이 정의한 문제와 솔루션을 SNS에 공개한 게시물의 링크. 이해관계 없는 사용자에게 문제에 대한 공감과 솔루션에 대한 반응을 얻고, 그 반응이 서비스 개발사에 제안할 근거가 된다.

형식은 사용자의 반응을 가장 잘 끌어내는 쪽으로 직접 정한다. 지인만 보는 계정보다 낯선 사람에게까지 닿는 채널이 낫다. 전문가가 선정한 게시물은 Phi 인스타그램이나 전문가의 SNS 채널에 게시될 수 있다.

F. 루브릭

이 섹션은 제출한 과제에 대해 셀프·피어·전문가 피드백을 주고받을 때의 공통 기준이다. Phi에서 피드백은 과제를 평가하는 일이 아니라 학습을 견인하는 메타인지 장치이며, 루브릭은 이 세 층의 피드백이 같은 기준 위에서 이루어지도록 한다. Learning Outcome(LO)은 학습 후 도달하는 상태로, C의 After 1~4와 같다. Observable Evidence(OE, 관찰 가능한 증거)는 과제에서 LO 달성을 확인할 수 있는 신호다. OE가 모두 관찰되는 과제는 LO를 달성한 과제라고 할 수 있다. 매주 과제를 제출하기 전 스스로 점검할 때, 그리고 피드백을 주고받을 때 이 섹션을 참고한다.

LO1: 사용자의 불편함을 예민하게 감지하고, 그것이 누구에게 어떤 상황에서 일어나는지 장면 단위로 생생하게 서술할 수 있다.

OE 1: 불편함이 특정 사용자의 특정 순간으로 적혀 있다. 언제, 어디서, 무엇을 하다가 겪는지가 문장에서 드러난다.

OE 2: 문제가 실제 사용자의 발화나 사건에서 출발했다는 흔적이 있다.

OE 3: 불편을 방치하면 무엇을 잃는지가 상황별로 적혀 있다. '불편하다'라는 말로 뭉뚱그려져 있지 않다.

OE 4: 사용자가 목표를 달성하기 위해 현재 하고 있는 우회 행동이 적혀 있다.

OE 5: 배경지식이 없는 사람도 1 Pager만 읽고 장면을 그릴 수 있다.

LO2: 증상과 문제를 구분하고, 그 문제가 발생하는 근본 원인(root cause)이 무엇인지 논리적으로 설명할 수 있다.

OE 1: 표면 현상과 근본 원인이 구분되어 계층 구조로 적혀 있다.

OE 2: 원인이 한 갈래가 아니라 복수 후보로 탐색된 뒤, 그중 하나가 선택되어 있다.

OE 3: 니즈에서 원인까지 why의 사슬이 끊기지 않고 이어진다.

OE 4: 선택되지 않은 원인이 있다면, 다루지 않는 이유가 표시되어 있다.

F. 루브릭

이 섹션은 제출한 과제에 대해 셀프·피어·전문가 피드백을 주고받을 때의 공통 기준이다. Phi에서 피드백은 과제를 평가하는 일이 아니라 학습을 견인하는 메타인지 장치이며, 루브릭은 이 세 층의 피드백이 같은 기준 위에서 이루어지도록 한다.

Learning Outcome(LO)은 학습 후 도달하는 상태로, C의 After 1~4와 같다. Observable Evidence(OE, 관찰 가능한 증거)는 과제에서 LO 달성을 확인할 수 있는 신호다. OE가 모두 관찰되는 과제는 LO를 달성한 과제라고 할 수 있다. 매주 과제를 제출하기 전 스스로 점검할 때, 그리고 피드백을 주고받을 때 이 섹션을 참고한다.

LO3: 문제와 정확히 연결되어, 사용자가 쓰는 순간 문제가 풀렸다고 느끼는 해결책을 만들 수 있다.

OE 1: 솔루션의 각 장치가 원인의 특정 지점과 1:1로 대응한다. 원인에 없는 장치, 장치 없는 원인이 없다.

OE 2: 이미 존재하는 솔루션이 있다면, 그것이 이 원인을 해결하지 못하는 이유가 적혀 있다.

OE 3: 채택하지 않은 대안과 기각 근거가 남아 있다. 기각 근거가 사용자의 특성이나 맥락에 닿아 있다.

OE 4: 사용자가 추측, 재시도, 수동 작업으로 스스로 메우던 일을 솔루션이 대신한다. 새 과업을 부과한다면, 그 과업으로 없어지는 비용이 생기는 비용보다 크다는 근거가 있다.

LO4: 프로토타입으로 문제가 실제로 해결되는지 검증하고, 그 결과에서 더 근본적인 원인을 발견해 문제를 다시 정의할 수 있다.

OE 1: 무엇이 관찰되면 해결로 보는지, 판정 기준이 검증 전에 먼저 적혀 있다. 그 기준은 사용자의 말이 아니라 행동이다.

OE 2: 검증 결과에 따른 다음 개선 계획과 그 근거가 버전 로그에 기록되어 있다.

OE 3: 검증 결과에서 더 근본적인 원인을 발견해 문제를 재정의한 사이클이 최소 1회 있다.

OE 4: 솔루션이 새 문제를 만들지 않는지 확인하는 가드레일 항목이 있다.

OE 5: 재정의된 새 문제가 이전 문제의 원인 계보에서 이어져 있다.